



وزارة التحول الرقمي وعصرنة الإدارة  
Ministère de la Transformation Numérique  
et de la Modernisation de l'Administration



**Université de Nouakchott**

Faculté des Sciences et Techniques

Département d'Informatique

# MÉMOIRE DE FIN D'ÉTUDES

Pour l'obtention du diplôme de Licence Professionnelle en  
**Méthodes Informatiques Appliquées à la Gestion des  
Entreprises (MIAGE)**

**Sujet :**

**Plateforme de gestion des réclamations IT et du  
matériel informatique**

**Présenté par :**

Mouna Naji

**Encadré par :**

Mr. Beide Bouh Chekhne (FST)

M.Babah Ivikou

**Date de soutenance : ..... / ..... / 2026**

**Année universitaire : 2025 – 2026**

# Dédicace

## **À mes enseignants,**

En reconnaissance de tout ce qu'ils m'ont apporté en savoir, conseils et orientation, leurs paroles ont été une lumière qui a éclairé mon chemin.

## **À mes encadrants de projet,**

Merci pour leur présence constante et leur soutien sincère, qui ont constitué un véritable appui à chaque étape de cette formation.

## **À mes collègues et camarades de la formation,**

Pour avoir fait de ce parcours une expérience riche en collaboration, en soutien et en moments inoubliables.

## **À toute personne qui a cru en moi et m'a soutenu,**

Par un simple mot ou un encouragement, leur impact a été important dans la réalisation de ce travail.

# Remerciements

Je tiens à exprimer ma profonde gratitude à toutes les personnes ayant contribué à la réussite de ce projet, que ce soit de manière directe ou indirecte, à travers leur soutien, leur aide et leurs encouragements tout au long de cette expérience.

Je souhaite adresser mes remerciements les plus sincères au **Mr. Beide Bouh Chekhne**, encadrant académique de ce travail, pour la précision de ses orientations, la pertinence de ses remarques ainsi que son encadrement continu qui a grandement participé à la réalisation de ce mémoire.

Je remercie également **M.Babah Ivikou**, encadrant professionnel au sein du **Ministère de la Transformation Numérique, de l'Innovation et de la Modernisation de l'Administration (MTNIMA)**, pour m'avoir permis d'intégrer un environnement professionnel réel, favorisant l'application concrète de mes connaissances.

J'exprime aussi ma reconnaissance envers les enseignants ainsi que l'administration **Faculté des Sciences et Techniques (FST)** pour la qualité de leur encadrement, la solidité de la formation dispensée et leur engagement à offrir des conditions d'apprentissage favorables.

Je n'oublie pas de remercier les membres du jury pour le temps consacré à l'évaluation de ce travail, l'intérêt porté à son contenu ainsi que la pertinence de leurs observations constructives.

Enfin, j'adresse mes remerciements à mes collègues et amis pour leur soutien constant, leur collaboration et les moments partagés qui ont rendu cette expérience plus enrichissante.

# Résumé

Ce projet vise à développer une plateforme web de gestion des réclamations liées aux technologies de l'information et au matériel informatique, afin d'améliorer l'enregistrement, le traitement et le suivi des incidents au sein de l'administration publique. La plateforme fournit un système centralisé permettant aux agents de soumettre des réclamations, tout en permettant au service informatique de les suivre et de les traiter de manière organisée et efficace. Le système repose sur l'analyse des besoins et la conception d'un modèle adapté, puis son développement à l'aide de technologies web modernes, accompagné de tests fonctionnels. Les premiers tests montrent que le système est efficace et facile à utiliser, et qu'il améliore la qualité et la rapidité du traitement des réclamations techniques.

# Table des matières

|                                                                     |           |
|---------------------------------------------------------------------|-----------|
| Dédicace . . . . .                                                  | i         |
| Remerciements . . . . .                                             | ii        |
| Résumé . . . . .                                                    | iii       |
| Liste des figures . . . . .                                         | vi        |
| Liste des tableaux . . . . .                                        | vii       |
| Liste des abréviations . . . . .                                    | viii      |
| <b>Introduction Générale . . . . .</b>                              | <b>1</b>  |
| Contexte du projet . . . . .                                        | 1         |
| Problématique . . . . .                                             | 1         |
| Objectifs du projet . . . . .                                       | 1         |
| Importance du projet . . . . .                                      | 2         |
| Travaux antérieurs . . . . .                                        | 2         |
| Méthodologie adoptée . . . . .                                      | 2         |
| Organisation du rapport . . . . .                                   | 3         |
| <b>I Analyse des besoins . . . . .</b>                              | <b>4</b>  |
| I.1 Présentation générale . . . . .                                 | 4         |
| I.2 Besoins fonctionnels . . . . .                                  | 5         |
| I.3 Besoins non-fonctionnels . . . . .                              | 5         |
| <b>II Conception . . . . .</b>                                      | <b>7</b>  |
| II.1 Présentation de l'UML . . . . .                                | 7         |
| II.2 Diagramme de cas d'utilisation . . . . .                       | 7         |
| II.3 Diagramme de classes . . . . .                                 | 13        |
| II.3.1 Authentification (Login JWT) . . . . .                       | 15        |
| II.3.2 Création d'une réclamation (Agent) . . . . .                 | 16        |
| II.3.3 Traitement d'une réclamation (Admin et Technicien) . . . . . | 17        |
| II.4 Diagramme d'états-transitions . . . . .                        | 18        |
| <b>III Technologies utilisées . . . . .</b>                         | <b>20</b> |
| III.1 Langages de programmation . . . . .                           | 20        |
| III.1.1 Java . . . . .                                              | 20        |
| III.1.2 JavaScript . . . . .                                        | 20        |
| III.1.3 HTML5 et CSS3 . . . . .                                     | 21        |
| III.2 Frameworks et Bibliothèques . . . . .                         | 21        |
| III.2.1 Spring Boot . . . . .                                       | 21        |
| III.2.2 React . . . . .                                             | 21        |
| III.2.3 Axios . . . . .                                             | 22        |
| III.2.4 React Router . . . . .                                      | 22        |

|                                                              |           |
|--------------------------------------------------------------|-----------|
| III.2.5 i18next . . . . .                                    | 22        |
| III.3 Stockage des données . . . . .                         | 22        |
| III.3.1 MySQL . . . . .                                      | 22        |
| III.4 Envoi d'e-mails . . . . .                              | 22        |
| III.5 Système de notifications . . . . .                     | 23        |
| III.6 Gestion de versions . . . . .                          | 23        |
| III.6.1 Git / GitHub . . . . .                               | 23        |
| III.7 Logiciels et environnements de développement . . . . . | 23        |
| III.7.1 Visual Studio Code . . . . .                         | 23        |
| III.7.2 IntelliJ IDEA . . . . .                              | 23        |
| III.7.3 Bruno . . . . .                                      | 24        |
| III.8 Outils de construction . . . . .                       | 24        |
| III.8.1 Maven . . . . .                                      | 24        |
| III.8.2 Vite . . . . .                                       | 24        |
| III.9 Récapitulatif de l'environnement logiciel . . . . .    | 25        |
| <b>IV Réalisation du projet . . . . .</b>                    | <b>26</b> |
| IV.1 Architecture générale . . . . .                         | 26        |
| IV.2 Interfaces graphiques . . . . .                         | 27        |
| IV.3 Interface Admin . . . . .                               | 27        |
| IV.4 Interface Technicien . . . . .                          | 28        |
| IV.5 Interfaces Agent . . . . .                              | 29        |
| <b>Conclusion Générale . . . . .</b>                         | <b>31</b> |
| <b>Références bibliographiques . . . . .</b>                 | <b>32</b> |
| <b>Annexes . . . . .</b>                                     | <b>33</b> |
| Annexe A — Diagramme entité-association . . . . .            | 33        |
| Annexe B — Modèle logique des données . . . . .              | 33        |
| Annexe C — Guide d'installation . . . . .                    | 34        |
| Annexe D — Extrait du code source . . . . .                  | 35        |
| Annexe E — Attestation de stage . . . . .                    | 36        |
| Annexe F — Comptes personnels . . . . .                      | 37        |

# Table des figures

|        |                                                                                                                            |    |
|--------|----------------------------------------------------------------------------------------------------------------------------|----|
| I.1    | Organigramme du Ministère de la Transition Numérique, de l'Innovation et de la Modernisation de l'Administration . . . . . | 4  |
| I.2    | Structure Organisationnelle de la DAS . . . . .                                                                            | 5  |
| II.1   | Diagramme des cas d'utilisation-Admin . . . . .                                                                            | 9  |
| II.2   | Diagramme des cas d'utilisation-Agent . . . . .                                                                            | 11 |
| II.3   | Diagramme des cas d'utilisation-technicien . . . . .                                                                       | 13 |
| II.4   | Diagramme de classe . . . . .                                                                                              | 14 |
| II.5   | Diagramme de séquence - Authentification . . . . .                                                                         | 16 |
| II.6   | Diagramme de séquence - Création d'une réclamation . . . . .                                                               | 17 |
| II.7   | Diagramme de séquence - Traitement d'une réclamation . . . . .                                                             | 18 |
| II.8   | Diagramme d'états-transitions — Cycle de vie d'une réclamation . . .                                                       | 18 |
| III.1  | Logo Java . . . . .                                                                                                        | 20 |
| III.2  | Logo JavaScript . . . . .                                                                                                  | 21 |
| III.3  | Logos CSS3 et HTML5 . . . . .                                                                                              | 21 |
| III.4  | Logo Spring Boot . . . . .                                                                                                 | 21 |
| III.5  | Logo React . . . . .                                                                                                       | 21 |
| III.6  | Logo MySQL . . . . .                                                                                                       | 22 |
| III.7  | Logo VS Code . . . . .                                                                                                     | 23 |
| III.8  | Logo IntelliJ IDEA . . . . .                                                                                               | 24 |
| III.9  | Logo Bruno . . . . .                                                                                                       | 24 |
| III.10 | Logo Maven . . . . .                                                                                                       | 24 |
| III.11 | Logo Vite . . . . .                                                                                                        | 24 |
| IV.1   | Page login . . . . .                                                                                                       | 27 |
| IV.2   | Page Tableau de bord . . . . .                                                                                             | 27 |
| IV.3   | Page Réclamations : . . . . .                                                                                              | 28 |
| IV.4   | Page Tableau de bord . . . . .                                                                                             | 28 |
| IV.5   | Page Tableau de bord . . . . .                                                                                             | 29 |
| IV.6   | Page Mes Réclamations . . . . .                                                                                            | 29 |
| IV.7   | Modèle Conceptuel des Données (MCD) — GesRecIT . . . . .                                                                   | 33 |

# Liste des tableaux

|       |                                                                      |    |
|-------|----------------------------------------------------------------------|----|
| II.1  | Récapitulatif des acteurs du système . . . . .                       | 8  |
| II.2  | Description des cas d'utilisation de l'Administrateur . . . . .      | 10 |
| II.3  | Description des cas d'utilisation de l'Agent . . . . .               | 12 |
| II.4  | Description des cas d'utilisation du Technicien . . . . .            | 13 |
| III.1 | Récapitulatif de l'environnement logiciel de la plateforme . . . . . | 25 |
| IV.1  | Tables principales de la base de données GesRecIT . . . . .          | 34 |

# Liste des abréviations

| Abréviation | Signification                                                                                        |
|-------------|------------------------------------------------------------------------------------------------------|
| API         | Application Programming Interface                                                                    |
| BCrypt      | Algorithme de hachage de mots de passe                                                               |
| CSS         | Cascading Style Sheets                                                                               |
| DAS         | Direction de l'Administration des Systèmes et de la Sécurité                                         |
| HTML        | HyperText Markup Language                                                                            |
| HTTP        | HyperText Transfer Protocol                                                                          |
| IT          | Information Technology (Technologies de l'Information)                                               |
| Java        | Langage de programmation orienté objet                                                               |
| JPA         | Java Persistence API                                                                                 |
| JSON        | JavaScript Object Notation                                                                           |
| JWT         | JSON Web Token                                                                                       |
| MIAGE       | Méthodes Informatiques Appliquées à la Gestion des Entreprises                                       |
| MTNIMA      | Ministère de la Transformation Numérique, de l'Innovation et de la Modernisation de l'Administration |
| MySQL       | My Structured Query Language                                                                         |
| PFE         | Projet de Fin d'Études                                                                               |
| React       | Bibliothèque JavaScript pour l'interface utilisateur                                                 |
| REST        | Representational State Transfer                                                                      |
| SGBD        | Système de Gestion de Base de Données                                                                |
| SPA         | Single Page Application                                                                              |
| SQL         | Structured Query Language                                                                            |
| UML         | Unified Modeling Language                                                                            |
| URL         | Uniform Resource Locator                                                                             |

# Introduction Générale

## Contexte du projet

Dans un monde où la transformation numérique devient incontournable, les organisations cherchent à optimiser leurs processus internes. C'est dans ce contexte que s'inscrit notre projet de fin d'études portant sur le développement d'une plateforme web de gestion des réclamations IT et du matériel informatique.

Au sein des administrations publiques, les agents utilisent de plus en plus les outils numériques dans leurs activités quotidiennes. Cependant, les incidents liés aux logiciels, aux accès, au réseau ou au matériel informatique sont souvent signalés de manière informelle, ce qui entraîne un manque de suivi, de traçabilité et d'efficacité dans leur traitement.

Afin de remédier à ces difficultés, cette plateforme vise à centraliser la gestion des réclamations IT en permettant aux agents de déclarer facilement les incidents, tout en offrant au service informatique des outils efficaces pour le suivi, l'attribution et le traitement des demandes.

## Problématique

La gestion des réclamations IT dans les administrations présente plusieurs limites : signalement non centralisé des incidents, perte d'informations, manque de traçabilité des interventions et difficulté de suivi des demandes. La communication entre les agents et le service informatique reste souvent inefficace et informelle. Il en résulte des retards dans le traitement des incidents, une mauvaise répartition des tâches entre les techniciens IT et une faible visibilité pour les utilisateurs sur l'état d'avancement de leurs réclamations.

D'où la nécessité d'un système centralisé, fiable et accessible, capable d'assurer une gestion fluide, structurée et efficace des incidents IT.

## Objectifs du projet

**Objectif général :** Développer une plateforme web centralisée et sécurisée pour la gestion des réclamations liées aux technologies de l'information et au matériel informatique au sein de l'administration publique.

**Objectifs spécifiques :**

- Permettre aux utilisateurs de signaler facilement les incidents liés aux logiciels, au réseau, aux accès ou au matériel informatique.
- Améliorer le suivi et la traçabilité des réclamations, de leur déclaration jusqu'à leur clôture.
- Faciliter l'attribution et le traitement des demandes de manière organisée et efficace par les techniciens IT.
- Renforcer la communication et l'interaction entre les utilisateurs et le service informatique.
- Fournir une interface adaptée à chaque rôle utilisateur (agent, technicien IT, responsable, administrateur).
- Produire des statistiques et des rapports pour le pilotage et l'aide à la décision.

## Importance du projet

Ce projet présente un intérêt à la fois pratique et académique. Sur le plan pratique, il améliore la gestion des réclamations IT grâce à une solution centralisée et efficace, réduit le temps de traitement des incidents et renforce la communication entre les utilisateurs et le service informatique. Sur le plan académique, il constitue une expérience formatrice complète en développement web, en architecture logicielle et en gestion des systèmes d'information.

## Travaux antérieurs

Plusieurs solutions de gestion des réclamations IT existent sur le marché, telles que GLPI, JIRA Service Management ou Freshdesk. Cependant, ces solutions sont souvent génériques, complexes à configurer ou coûteuses, et rarement adaptées aux besoins spécifiques des administrations publiques mauritaniennes.

Notre projet se distingue par sa simplicité d'utilisation, son adaptation au contexte de l'administration publique et à ses contraintes organisationnelles, ainsi que par le recours à des technologies web modernes telles que Spring Boot et React (Vite), garantissant de bonnes performances et une évolutivité satisfaisante.

## Méthodologie adoptée

Pour réaliser ce projet, nous avons adopté une méthodologie de développement itérative et incrémentale, organisée en plusieurs étapes complémentaires :

- **Étude de l'existant** : analyse des solutions existantes et identification des limites du système manuel en place.
- **Analyse des besoins** : recueil des besoins fonctionnels et non fonctionnels auprès des parties prenantes.
- **Conception** : modélisation du système à l'aide des diagrammes UML (cas d'utilisation, classes, séquence, états-transitions).
- **Choix technologique** : sélection de Spring Boot pour le backend et de React (Vite) pour le frontend.
- **Développement** : implémentation progressive des modules (authentification, gestion des utilisateurs, gestion des réclamations, tableau de bord).
- **Tests fonctionnels** : vérification du bon fonctionnement de la plateforme.
- **Rédaction** : documentation et rédaction du présent rapport.

## Organisation du rapport

Ce rapport est structuré en quatre chapitres, présentés comme suit :

- **Chapitre I — Analyse des besoins** : présentation du contexte institutionnel, identification des acteurs et des besoins fonctionnels et non fonctionnels du système.
- **Chapitre II — Conception** : modélisation du système à travers les principaux diagrammes UML : cas d'utilisation, séquence, classes et états-transitions.
- **Chapitre III — Technologies utilisées** : présentation des langages de programmation, des frameworks et des outils retenus pour la réalisation du projet, avec une justification des choix technologiques.
- **Chapitre IV — Réalisation** : présentation de l'architecture technique, des interfaces graphiques de l'application et des fonctionnalités réalisées pour chaque profil utilisateur.

Le rapport se clôture par une conclusion générale synthétisant les résultats obtenus, les limites du travail et les perspectives d'amélioration envisagées.

# Chapitre I

## Analyse des besoins

### I.1 Présentation générale

Dans le cadre de notre projet de fin d'études, nous avons intégré le Ministère de la Transformation Numérique, de l'Innovation et de la Modernisation de l'Administration (MTNIMA), une institution publique engagée dans la modernisation des services administratifs en Mauritanie. Notre stage s'est déroulé au sein du service informatique, où nous avons participé à la conception et au développement d'une plateforme web de gestion des réclamations IT. L'objectif principal de ce projet est de centraliser et digitaliser la gestion des incidents techniques afin d'améliorer leur traitement.

- L'organigramme ci-dessous présente la structure générale de ce ministère.

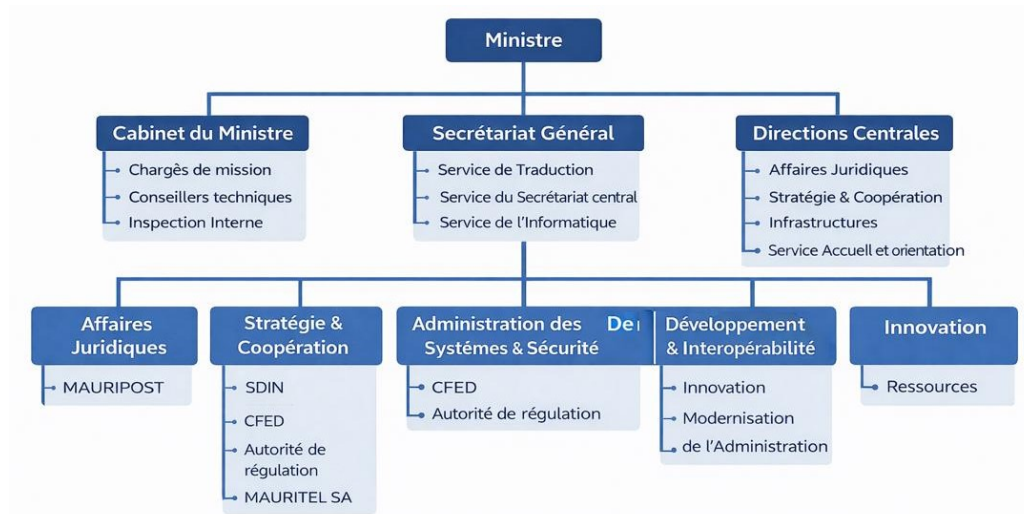


Fig. I.1 : Organigramme du Ministère de la Transition Numérique, de l'Innovation et de la Modernisation de l'Administration

La Direction de l'Administration des Systèmes et de la Sécurité (DAS) constitue le service au sein duquel ce projet a été réalisé. Elle est dirigée par un Directeur assisté d'un Directeur Adjoint, et comprend plusieurs divisions spécialisées, comme l'illustre la figure ci-dessous.

## Structure Organisationnelle de la DAS

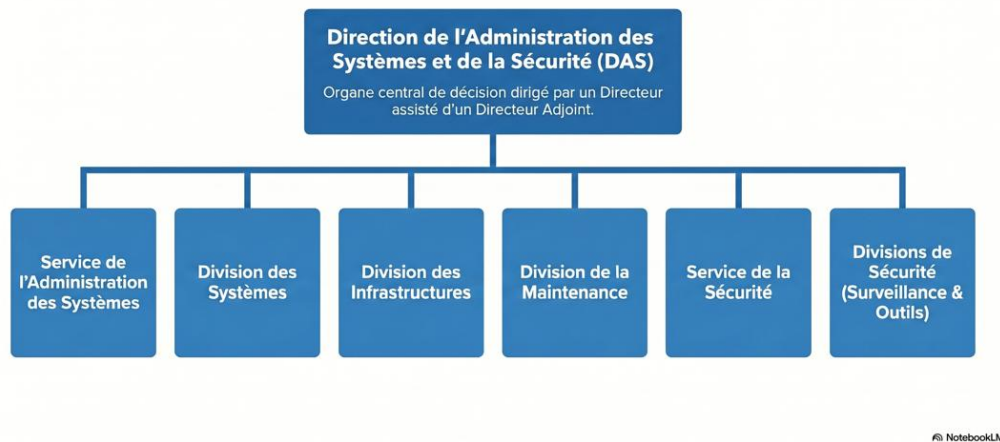


Fig. I.2 : Structure Organisationnelle de la DAS

## I.2 Besoins fonctionnels

Les besoins fonctionnels décrivent ce que le système doit faire. Voici les principales fonctionnalités identifiées :

- BF1.** Authentification et gestion multi-rôles (administrateur, technicien IT, agent).
- BF2.** Gestion des utilisateurs et des comptes.
- BF3.** Déclaration des réclamations liées aux logiciels, au réseau, aux accès et au matériel informatique.
- BF4.** Attribution des réclamations aux techniciens IT.
- BF5.** Suivi et traitement des incidents techniques.
- BF6.** Système de notifications et de commentaires.
- BF7.** Gestion du statut des réclamations.
- BF8.** Historique des actions et traçabilité des opérations.
- BF9.** Tableau de bord avec statistiques des réclamations.
- BF10.** Export des données et génération de rapports.

## I.3 Besoins non-fonctionnels

Les besoins non-fonctionnels définissent les critères de qualité et de performance du système :

- Sécurité : protection des données des utilisateurs et sécurisation des accès via authentification basée sur JWT. Les mots de passe ne sont jamais stockés en clair : ils sont hashés à l'aide de l'algorithme **BCrypt**, intégré nativement à Spring Security, qui applique un salage automatique rendant chaque hash unique et résistant aux attaques par table de correspondance.

- Ergonomie : interface simple et intuitive, pensée pour une utilisation sur ordinateur.

- Performance : optimisation du temps de réponse et efficacité des échanges entre le frontend et le backend, avec un temps de réponse cible inférieur à 2 secondes pour les opérations courantes (consultation, création d'une réclamation).

- Fiabilité : stabilité du système et gestion correcte des erreurs et exceptions.

- Scalabilité : architecture permettant l'ajout de nouvelles fonctionnalités sans impacter le système existant.

- Accessibilité : compatibilité avec les principaux navigateurs web modernes.

- Traçabilité : enregistrement des actions et des opérations pour assurer le suivi des réclamations et des utilisateurs.

# Chapitre II

## Conception

La phase de conception constitue une étape essentielle dans le développement de notre plateforme. Elle permet de traduire les besoins identifiés lors de l'analyse en modèles structurés facilitant l'implémentation du système. Dans ce chapitre, nous présentons les principaux diagrammes UML réalisés pour la conception de la plateforme de gestion des réclamations IT et du matériel informatique, notamment le diagramme de cas d'utilisation, les diagrammes de séquence et le diagramme de classes.

### II.1 Présentation de l'UML

L'UML (*Unified Modeling Language*) est un langage de modélisation graphique standardisé par l'OMG (*Object Management Group*), largement utilisé en génie logiciel pour la spécification, la visualisation, la construction et la documentation des systèmes informatiques. Il ne constitue pas une méthode de développement en soi, mais un ensemble de notations permettant de représenter les différentes facettes d'un système selon plusieurs points de vue complémentaires.

Dans le cadre de ce projet, nous distinguons deux grandes catégories de diagrammes :

- **Les diagrammes structurels**, notamment le *diagramme de classes*, qui décrit les entités du domaine métier, leurs attributs et les relations qui les unissent. Il a servi de base directe à la conception du schéma relationnel de la base de données MySQL.
- **Les diagrammes comportementaux**, regroupant le *diagramme de cas d'utilisation* (identification des fonctionnalités offertes à chaque acteur), le *diagramme de séquence* (échanges chronologiques entre les composants pour un scénario donné) et le *diagramme d'états-transitions* (cycle de vie d'un objet métier, ici une réclamation).

### II.2 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation permet d'identifier les interactions entre les différents acteurs et le système. Il offre une vue globale des fonctionnalités proposées par l'application. Dans notre système, les acteurs suivants sont présents :

- **Administrateur**
- **Technicien**
- **Agent**

| Acteur         | Profil                | Rôle principal                                                       |
|----------------|-----------------------|----------------------------------------------------------------------|
| Agent          | Utilisateur final     | Déclarer et suivre ses réclamations IT                               |
| Technicien     | Responsable technique | Traiter les interventions et résoudre les réclamations assignées     |
| Administrateur | Superviseur système   | Gérer les utilisateurs, les réclamations et le matériel informatique |

TAB. II.1 Récapitulatif des acteurs du système

Parmi les principaux cas d'utilisation, nous distinguons :

- **Authentification (connexion)**
- **Gestion des utilisateurs et des rôles**
- **Création et consultation des réclamations**
- **Attribution des réclamations aux techniciens IT**
- **Suivi et traitement des incidents**
- **Gestion du matériel informatique**
- **Réception des notifications**

Ces cas d'utilisation sont représentés à travers trois diagrammes distincts selon le rôle de l'acteur : Administrateur, Technicien IT et Agent.

Ce diagramme permet de mieux comprendre les responsabilités de chaque acteur ainsi que les services offerts par la plateforme.

- **Le diagramme de cas d'utilisation de l'Administrateur (Figure 2.1) illustre l'ensemble des fonctionnalités de gestion disponibles, incluant la gestion des utilisateurs, des réclamations et du matériel informatique.**

## Diagramme de cas d'utilisation-Admin

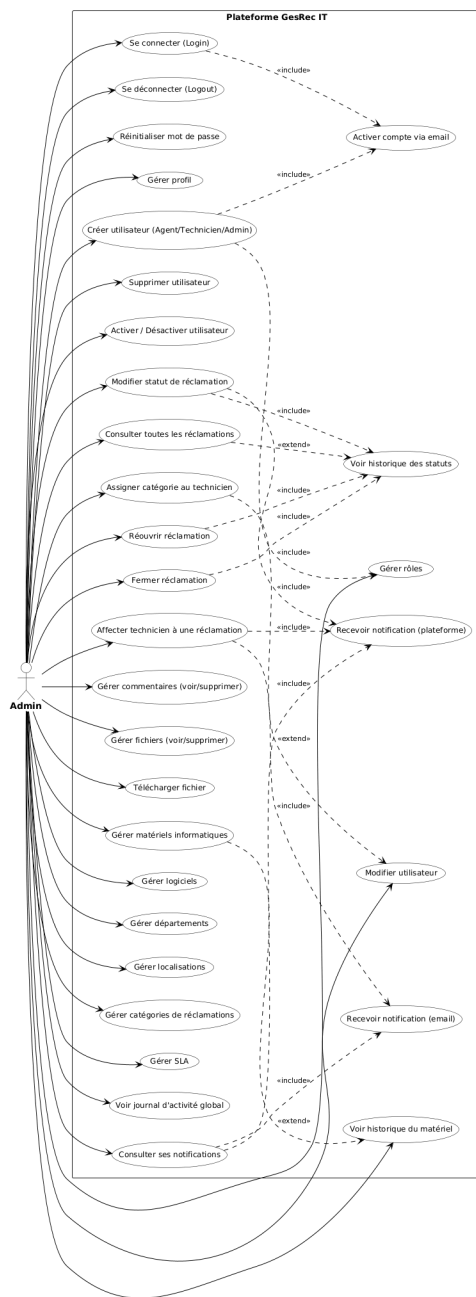


Fig. II.1 : Diagramme des cas d'utilisation-Admin

| Cas d'utilisation                     | Description                                                                              |
|---------------------------------------|------------------------------------------------------------------------------------------|
| Créer utilisateur                     | L'administrateur crée un compte (Agent, Technicien ou Admin) et lui attribue un rôle.    |
| Activer/Désactiver utilisateur        | L'administrateur active ou désactive l'accès d'un utilisateur à la plateforme.           |
| Consulter toutes les réclamations     | L'administrateur visualise l'ensemble des réclamations soumises par les agents.          |
| Affecter technicien à une réclamation | L'administrateur attribue une réclamation à un technicien pour traitement.               |
| Modifier statut de réclamation        | L'administrateur met à jour manuellement le statut d'une réclamation.                    |
| Fermer réclamation                    | L'administrateur clôture définitivement une réclamation résolue.                         |
| Relancer réclamation                  | L'administrateur réactive une réclamation nécessitant une intervention supplémentaire.   |
| Gérer matériels informatiques         | L'administrateur ajoute, modifie ou supprime les équipements du parc informatique.       |
| Gérer catégories de réclamations      | L'administrateur définit les catégories (Matériel, Logiciel, Réseau, Accès).             |
| Gérer SLA                             | L'administrateur configure les délais de résolution et de réponse par priorité.          |
| Voir journal d'activité global        | L'administrateur consulte l'historique complet des actions effectuées sur la plateforme. |

TAB. II.2 Description des cas d'utilisation de l'Administrateur

- Le diagramme de cas d'utilisation de l'Agent (Figure 2.2) illustre les interactions entre l'agent et le système, notamment la création et la consultation des réclamations.

## Diagramme de cas d'utilisation-Agent

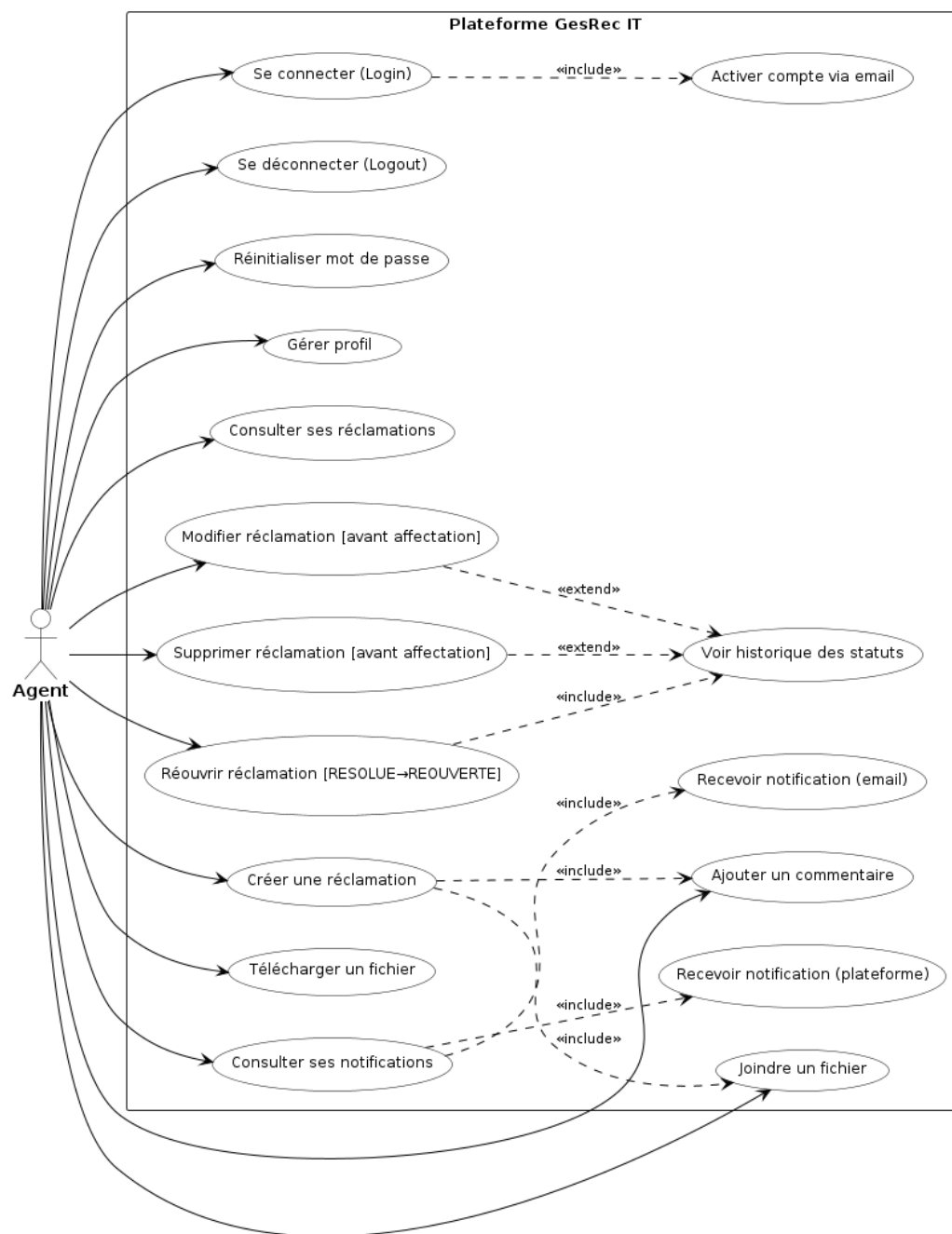


Fig. II.2 : Diagramme des cas d'utilisation-Agent

| Cas d'utilisation                         | Description                                                                       |
|-------------------------------------------|-----------------------------------------------------------------------------------|
| Créer une réclamation                     | L'agent déclare un incident lié au matériel, logiciel, réseau ou accès.           |
| Consulter ses réclamations                | L'agent visualise l'état de ses réclamations soumises.                            |
| Modifier réclamation [avant affectation]  | L'agent peut modifier une réclamation tant qu'aucun technicien n'y a été affecté. |
| Supprimer réclamation [avant affectation] | L'agent peut supprimer une réclamation avant son affectation à un technicien.     |
| Réouvrir réclamation [RESOLUE→REOUVERTE]  | L'agent réouvre une réclamation marquée comme résolue si le problème persiste.    |
| Joindre un fichier                        | L'agent ajoute une pièce jointe (image, document) à sa réclamation.               |
| Consulter ses notifications               | L'agent consulte les alertes reçues lors des changements de statut.               |

TAB. II.3 Description des cas d'utilisation de l'Agent

- Le diagramme de cas d'utilisation du Technicien (Figure 2.3) présente les fonctionnalités accessibles au technicien, telles que le traitement des interventions et la consultation des réclamations assignées.

## Diagramme de cas d'utilisation-technicien

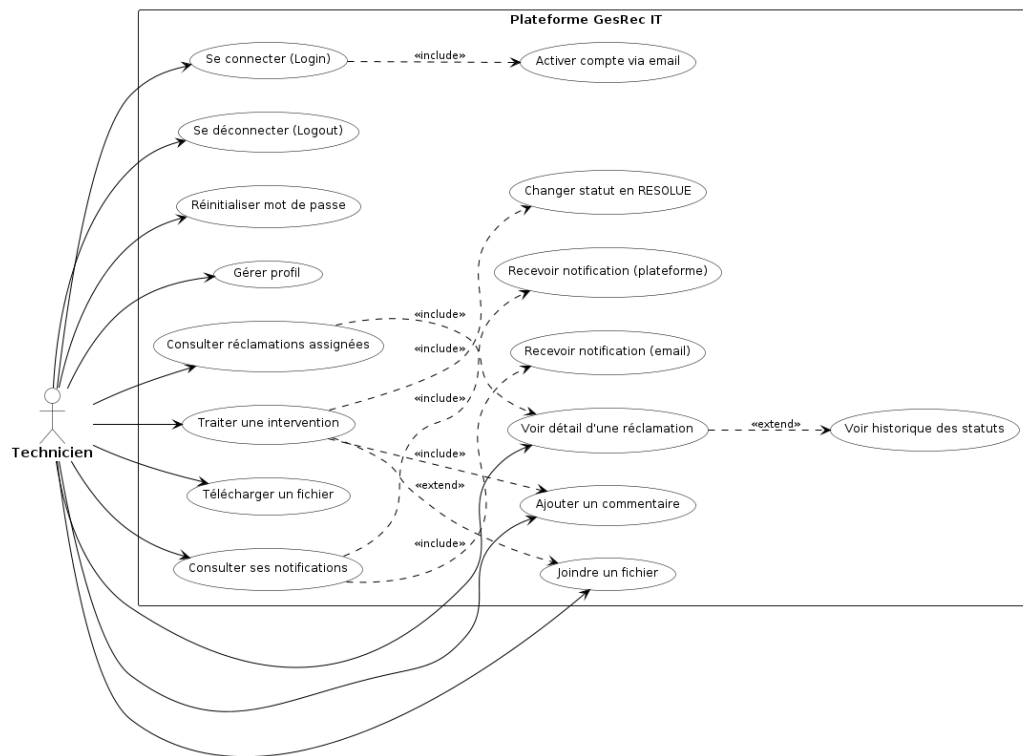


Fig. II.3 : Diagramme des cas d'utilisation-technicien

| Cas d'utilisation                | Description                                                                                    |
|----------------------------------|------------------------------------------------------------------------------------------------|
| Consulter réclamations assignées | Le technicien visualise les réclamations qui lui ont été attribuées.                           |
| Traiter une intervention         | Le technicien effectue les actions nécessaires à la résolution de l'incident.                  |
| Changer statut en RESOLUE        | Le technicien marque une réclamation comme résolue une fois l'intervention terminée.           |
| Voir détail d'une réclamation    | Le technicien consulte les informations complètes d'une réclamation, y compris son historique. |
| Ajouter un commentaire           | Le technicien documente son intervention par un commentaire.                                   |

TAB. II.4 Description des cas d'utilisation du Technicien

## II.3 Diagramme de classes

Le diagramme de classes (Figure II.4) représente la structure statique de l'application, en illustrant les principales entités du système ainsi que leurs attributs et les relations qui les lient, permettant ainsi de structurer le code source et de concevoir une base de données cohérente avec les besoins fonctionnels du système.



- **SLA** : définit les délais de résolution et de réponse selon le niveau de priorité (Faible, Moyenne, Haute, Critique).
- **Affectation** : trace l'attribution d'une réclamation à un technicien, avec la date et l'auteur de l'affectation.
- **Intervention** : décrit une action technique réalisée par un technicien sur une réclamation, avec son type, ses détails et ses dates de début/fin.
- **Commentaire** : représente un échange textuel entre les acteurs autour d'une réclamation.
- **PieceJointe** : représente un fichier joint à une réclamation (nom, type, taille, date de dépôt).
- **HistoriqueStatut** : conserve la trace de chaque changement de statut d'une réclamation (ancien statut, nouveau statut, auteur).
- **HistoriqueMateriel** : conserve l'historique des actions effectuées sur un matériel donné.
- **JournalActivite** : enregistre l'ensemble des actions effectuées sur la plateforme à des fins de traçabilité globale.
- **Verification (Notification)** : représente une alerte envoyée à un utilisateur, soit sur la plateforme, soit par e-mail, lors d'un événement système.

### II.3.1 Authentification (Login JWT)

Le diagramme de séquence (Figure II.5) illustre les interactions entre les objets du système lors de l'authentification d'un utilisateur. Pour le scénario de **connexion utilisateur avec génération de token JWT**, le scénario est le suivant :

1. L'utilisateur envoie une requête **POST /auth/login** avec son email et son mot de passe.
2. Le **AuthenticationController** transmet les identifiants à l'**AuthenticationService** pour vérification.
3. L'**AuthenticationService** vérifie les identifiants via le **UserDetailsService** et le **PasswordEncoder**.
4. Si les identifiants sont valides, le **JwtService** génère un token JWT (accessToken et refreshToken) et le renvoie à l'utilisateur avec un code **200 OK**.
5. Si les identifiants sont invalides, une exception est levée et une réponse **400 Bad Request** est renvoyée.

6. Pour les requêtes suivantes, le token est validé via le **JwtAuthenticationFilter** et le **SecurityContext**, permettant d'autoriser ou de rejeter (**401 Unauthorized**) la requête.

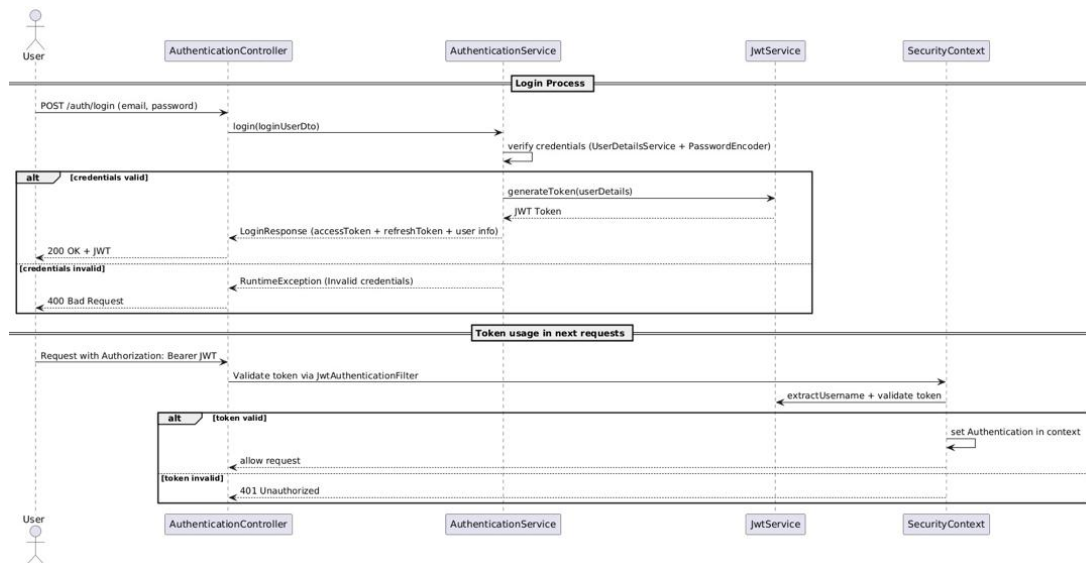


Fig. II.5 : Diagramme de séquence - Authentification

## Gestion du Refresh Token

Le système d'authentification repose sur deux jetons distincts : un **accessToken**, valable **1 heure**, utilisé pour authentifier chaque requête API, et un **refreshToken**, valable **7 jours**, permettant de générer un nouvel accessToken sans exiger une nouvelle saisie des identifiants par l'utilisateur. Cette durée de vie courte de l'accessToken limite la fenêtre d'exposition en cas de compromission du jeton, tandis que le refreshToken assure une expérience utilisateur fluide en maintenant la session active sur une durée plus longue.

### II.3.2 Création d'une réclamation (Agent)

Le diagramme de séquence (Figure II.6) illustre les interactions entre les objets du système lors de la création d'une réclamation. Pour le scénario de **création et dépôt d'une réclamation par un agent**, le scénario est le suivant :

1. L'agent envoie une requête **POST /agent/reclamations** avec les données de la réclamation.
2. L'**AgentController** récupère l'utilisateur authentifié via le **SecurityContext**.
3. L'**AgentReclamationService** crée une nouvelle réclamation et la sauvegarde via le **ReclamationRepository**.
4. La réclamation est insérée dans la **Database**, qui renvoie l'entité sauvegardée avec son identifiant généré.

5. Le système renvoie une réponse **200 OK** avec le **ReclamationResponseDto** à l'agent.
6. Optionnellement, l'agent peut joindre un fichier via **POST /agent/reclamations/{id}/fichiers**, traité par le **PieceJointeService** qui stocke les métadonnées du fichier.

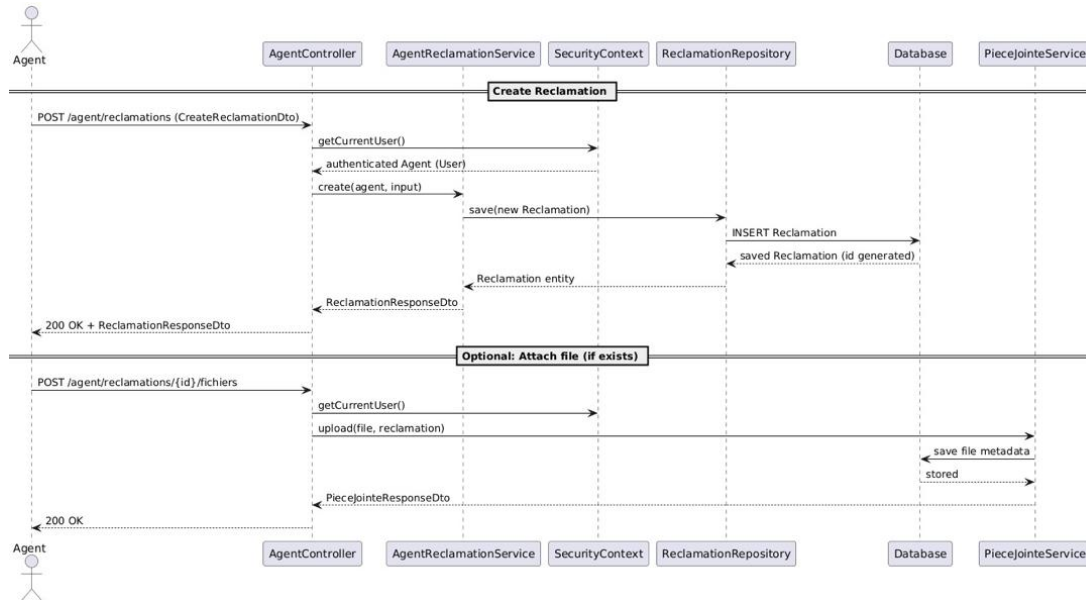


Fig. II.6 : Diagramme de séquence - Création d'une réclamation

### II.3.3 Traitement d'une réclamation (Admin et Technicien)

Le diagramme de séquence (Figure II.7) illustre les interactions entre les objets du système lors du traitement d'une réclamation. Pour le scénario **d'affectation et de résolution d'une réclamation**, le scénario est le suivant :

1. L'admin envoie une requête **PATCH /reclamations/{id}/assign/{technicienId}** pour affecter un technicien.
2. L'**AdminController** transmet la demande à l'**AdminService**, qui récupère la réclamation via le **ReclamationRepository** et met à jour le champ *technicien\_id* en base de données.
3. Le technicien résout ensuite la réclamation, déclenchant une mise à jour de statut (**RESOLUE**) via le **TechnicienService**.
4. La réclamation mise à jour est renvoyée avec un code **200 OK**.
5. Optionnellement, le technicien peut créer un rapport d'intervention via **POST /reclamations/{id}/intervention**, enregistré dans la base de données via le **TechnicienController**.

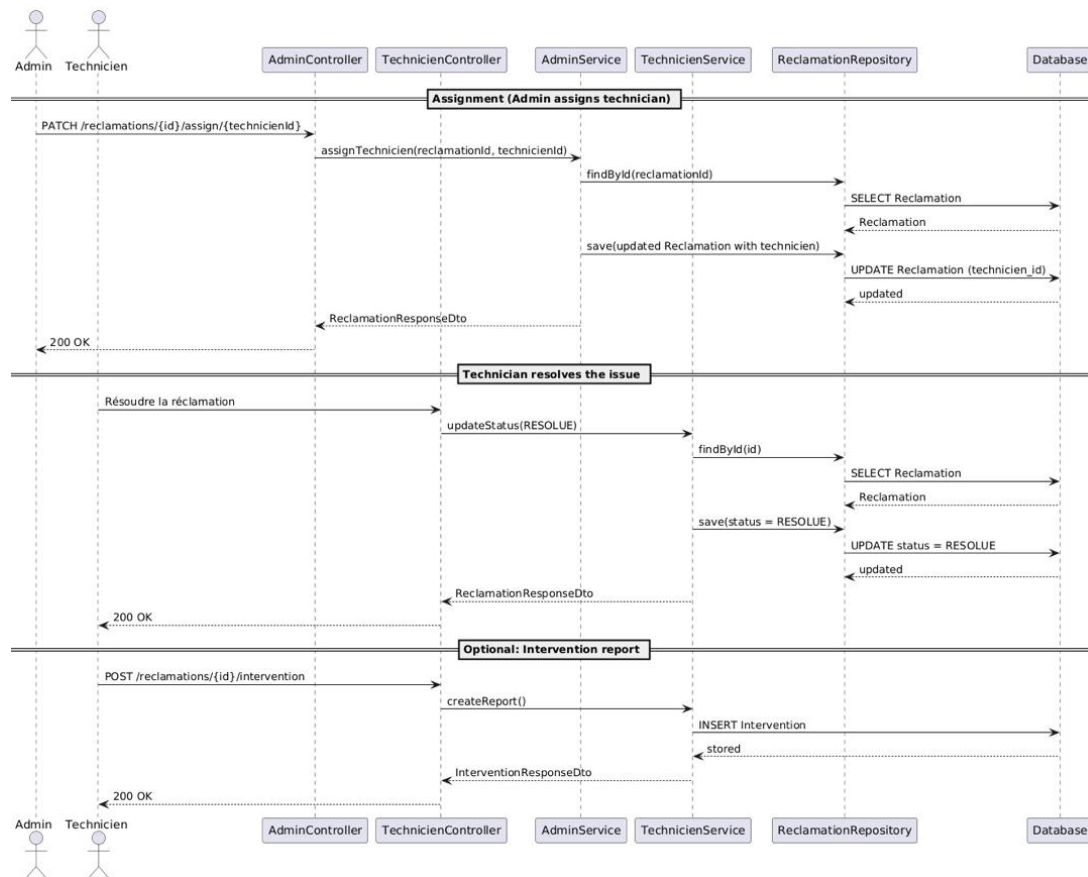


Fig. II.7 : Diagramme de séquence - Traitement d'une réclamation

## II.4 Diagramme d'états-transitions

Le diagramme d'états-transitions modélise le cycle de vie complet d'une réclamation, depuis sa création par l'agent jusqu'à son état final. Il met en évidence les transitions autorisées entre les statuts et les conditions qui les déclenchent.

Une réclamation naît au statut **En attente**. Dès qu'un technicien lui est affecté, elle passe à **En cours**. Une fois l'intervention terminée, le technicien la marque comme **Résolue**. L'administrateur peut alors la clôturer définitivement (**Fermée**), ou l'agent peut la réouvrir si le problème persiste, la faisant repasser à l'état **Réouverte** puis de nouveau **En cours**. Par ailleurs, tant qu'aucun technicien n'a été affecté, l'agent conserve la possibilité de modifier ou de supprimer sa réclamation, ou de l'annuler.

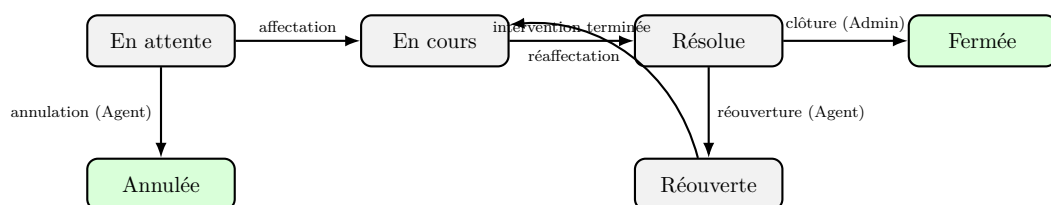


Fig. II.8 : Diagramme d'états-transitions — Cycle de vie d'une réclamation

## Conclusion

Ce chapitre nous a permis de modéliser de manière formelle et structurée les besoins fonctionnels de la plateforme à travers les principaux diagrammes UML. L'identification des acteurs, la description des cas d'utilisation, la conception du diagramme de classes, la modélisation des scénarios de séquence et la formalisation du cycle de vie d'une réclamation par le diagramme d'états-transitions constituent ensemble une base de conception solide et cohérente. Ces modèles ont servi de référence directe pour la phase de réalisation, détaillée dans le chapitre suivant.

# Chapitre III

## Technologies utilisées

### Introduction

Ce chapitre met en lumière les différentes ressources techniques et logicielles qui ont été employées lors du développement de l'application de gestion du parc informatique et du support technique. Il expose les choix technologiques effectués et démontre la maîtrise des outils utilisés tout au long du projet.

L'objectif est de présenter les langages de programmation, les environnements de développement, les bibliothèques et les logiciels qui ont été sélectionnés pour concevoir et mettre en œuvre cette application. Chaque outil a été choisi pour sa pertinence, sa facilité d'intégration et sa capacité à répondre aux besoins spécifiques du projet.

En mettant en avant ces outils, ce chapitre illustre la démarche méthodique adoptée et la volonté d'offrir une expérience utilisateur fluide, sécurisée et performante.

### III.1 Langages de programmation

#### III.1.1 Java

**Java** est un **langage de programmation orienté objet** utilisé principalement pour le développement du backend de l'application. Il permet de créer des applications robustes, sécurisées et adaptées aux systèmes d'information complexes. Dans ce projet, Java a été utilisé avec le framework Spring Boot pour implémenter la logique métier et la gestion des données.



Fig. III.1 : Logo Java

#### III.1.2 JavaScript

**JavaScript** est un **langage de programmation** utilisé côté frontend pour rendre l'application dynamique et interactive. Il a été utilisé avec React afin de développer une interface utilisateur moderne et réactive.



Fig. III.2 : Logo JavaScript

### III.1.3 HTML5 et CSS3

HTML5 a servi à structurer les pages web de l'application, tandis que CSS3 a permis de définir leur style et leur présentation visuelle.



Fig. III.3 : Logos CSS3 et HTML5

## III.2 Frameworks et Bibliothèques

### III.2.1 Spring Boot

**Spring Boot** est le **framework** principal utilisé pour le développement du backend. Il permet de créer des API REST et de structurer efficacement l'application.



Fig. III.4 : Logo Spring Boot

### III.2.2 React

**React** est une **bibliothèque JavaScript** utilisée pour construire l'interface utilisateur sous forme de composants réutilisables. La gestion de l'état de l'application repose sur les **Context API** natifs de React, sans recours à une bibliothèque externe de gestion d'état (comme Redux), ce qui permet de partager les données globales (utilisateur connecté, notifications, etc.) entre les différents composants de manière simple et légère.

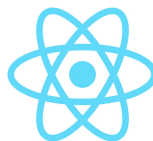


Fig. III.5 : Logo React

### III.2.3 Axios

**Axios** est une bibliothèque utilisée pour effectuer les appels HTTP vers l'API REST du backend. Elle simplifie l'envoi des requêtes et le traitement des réponses au format JSON, tout en permettant la configuration d'intercepteurs pour joindre automatiquement le jeton JWT à chaque requête.

### III.2.4 React Router

**React Router** assure la navigation et le routage entre les différentes pages de l'application (*Single Page Application*), notamment la gestion des routes protégées selon le rôle de l'utilisateur connecté (Admin, Technicien, Agent).

### III.2.5 i18next

Afin de proposer une interface multilingue (français, anglais, arabe), la plateforme intègre la bibliothèque **i18next**, accompagnée de **react-i18next** pour son intégration native avec React. Cette solution permet de traduire dynamiquement l'ensemble des textes de l'interface selon la langue sélectionnée par l'utilisateur, comme illustré par le sélecteur de langue (FR/EN/AR) visible sur l'ensemble des pages de l'application.

## III.3 Stockage des données

### III.3.1 MySQL

**MySQL** est un système de gestion de base de données relationnelle **SGBDR** utilisé pour stocker les données de l'application. Il a été choisi pour sa fiabilité, ses performances et sa parfaite intégration avec Spring Boot.



Fig. III.6 : Logo MySQL

## III.4 Envoi d'e-mails

L'envoi des e-mails est assuré via le module **Spring Boot Mail**, à travers un serveur SMTP (Gmail). Ce service est utilisé pour :

- l'envoi d'un lien d'activation lors de la création d'un compte utilisateur ;

- l'envoi d'un lien de réinitialisation du mot de passe ;
- l'envoi de notifications par e-mail lors des changements de statut d'une réclamation.

## III.5 Système de notifications

La plateforme intègre un système de notifications à deux niveaux : des notifications affichées directement dans l'interface (plateforme) et des notifications envoyées par e-mail. Ces notifications informent les utilisateurs des événements clés liés à leurs réclamations (création, changement de statut, affectation à un technicien, ajout de commentaire).

## III.6 Gestion de versions

### III.6.1 Git / GitHub

Le versionnement du code source du projet est assuré par **Git**, avec un dépôt distant hébergé sur **GitHub**, permettant le suivi de l'historique des modifications et la sauvegarde du code source tout au long du développement.

## III.7 Logiciels et environnements de développement

### III.7.1 Visual Studio Code

**Visual Studio Code (VS Code)** est l'environnement de développement utilisé pour le développement du frontend de l'application. Il a permis d'écrire, d'organiser et de maintenir le code source de manière efficace grâce à ses nombreuses extensions dédiées au développement web, notamment pour React, HTML, CSS et JavaScript.



Fig. III.7 : Logo VS Code

### III.7.2 IntelliJ IDEA

**IntelliJ IDEA** est utilisé pour le développement du backend avec Java et Spring Boot.



Fig. III.8 : Logo IntelliJ IDEA

### III.7.3 Bruno

**Bruno** est un outil de test d'API REST utilisé pour tester les endpoints du backend, notamment l'authentification, la gestion des rôles et la validité des réponses HTTP.



Fig. III.9 : Logo Bruno

## III.8 Outils de construction

### III.8.1 Maven

**Maven** est un outil de gestion de projet et de dépendances utilisé pour le backend Java.



Fig. III.10 : Logo Maven

### III.8.2 Vite

**Vite** est un outil de développement utilisé pour le frontend React afin de faciliter le développement et le build de l'application.



Fig. III.11 : Logo Vite

## III.9 Récapitulatif de l’environnement logiciel

| Outil / Technologie | Version | Rôle                                                |
|---------------------|---------|-----------------------------------------------------|
| Visual Studio Code  | —       | Éditeur de code (frontend)                          |
| IntelliJ IDEA       | —       | Éditeur de code (backend)                           |
| Java                | 17      | Langage du backend                                  |
| Spring Boot         | 3.5.7   | Framework backend                                   |
| Maven               | —       | Build et gestion des dépendances                    |
| JWT                 | —       | Authentification (accessToken 1h / refreshToken 7j) |
| BCrypt              | —       | Hashage des mots de passe                           |
| React               | 19.2.4  | Framework frontend SPA                              |
| Axios               | —       | Appels HTTP vers l’API REST                         |
| React Router        | —       | Navigation et routage SPA                           |
| i18next             | —       | Internationalisation (FR/EN/AR)                     |
| Vite                | 8.0.4   | Build et développement frontend                     |
| MySQL               | 8.0.45  | Base de données relationnelle                       |
| Spring Boot Mail    | —       | Envoi d’e-mails (SMTP Gmail)                        |
| Git / GitHub        | —       | Versionnement et hébergement du code                |
| Bruno               | —       | Test des API REST                                   |

TAB. III.1 Récapitulatif de l’environnement logiciel de la plateforme

## Conclusion

Ce chapitre a permis de présenter les principales technologies et outils utilisés pour la réalisation de l’application de gestion du parc informatique et du support technique. Le choix de technologies modernes telles que Java, Spring Boot, React et MySQL a contribué à la conception d’une application performante, sécurisée et évolutive.

Par ailleurs, l’utilisation d’outils de développement et de gestion tels que Visual Studio Code, IntelliJ IDEA, Maven, Vite, Git et Bruno a facilité les différentes phases de développement, de test et de maintenance du projet. L’ensemble de ces technologies a permis de répondre efficacement aux besoins fonctionnels et techniques définis au début du projet.

# Chapitre IV

## Réalisation du projet

### Introduction

Ce chapitre décrit en détail la phase de réalisation du système. Après avoir défini les besoins fonctionnels et techniques, et conçu l'architecture du système, l'étape d'implémentation a permis de transformer ces spécifications en une application web fonctionnelle. Nous présentons ici les différentes interfaces du système, les fonctionnalités associées à chaque rôle utilisateur, ainsi que les technologies utilisées pour construire l'interface et gérer la logique métier. Des captures d'écran des principales pages de l'application seront également intégrées afin d'illustrer visuellement les étapes de développement.

### IV.1 Architecture générale

La plateforme repose sur une architecture en trois couches (*3-tiers*) :

- **Couche présentation** : une application *Single Page Application* développée en React, exécutée côté client et communiquant avec le backend via des appels HTTP (Axios) au format JSON.
- **Couche métier** : une API REST développée avec Spring Boot, organisée selon le pattern *Controller* → *Service* → *Repository*, exposant les fonctionnalités du système et appliquant les règles de sécurité (JWT, contrôle des rôles).
- **Couche données** : une base de données relationnelle MySQL, gérée via Hibernate/JPA, assurant la persistance des entités du système (utilisateurs, réclamations, matériels, etc.).

Cette séparation en couches favorise la maintenabilité, l'évolutivité et la testabilité indépendante de chaque partie du système.

## IV.2 Interfaces graphiques

Page login :

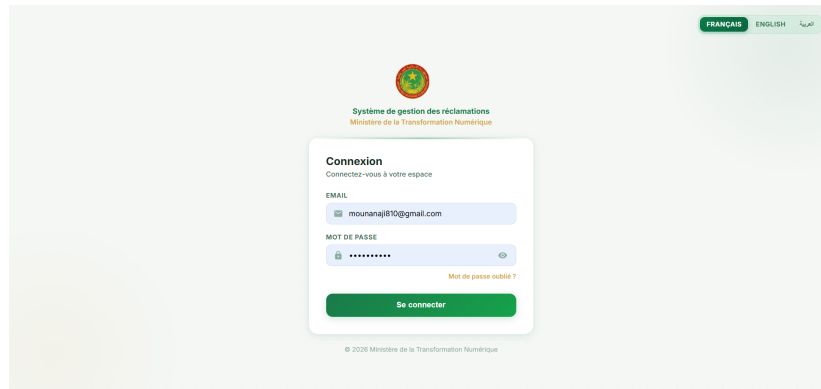


Fig. IV.1 : Page login

Cette capture illustre l'interface d'authentification de la plateforme GesRecIT. Elle présente un formulaire simple comprenant les champs e-mail et mot de passe, ainsi qu'un sélecteur de langue (FR/EN/AR) témoignant du caractère multilingue de l'application. Le design épuré, associé à l'emblème du Ministère, renforce l'identité institutionnelle du système tout en garantissant une prise en main intuitive pour l'ensemble des utilisateurs.

## IV.3 Interface Admin

Page Tableau de bord :

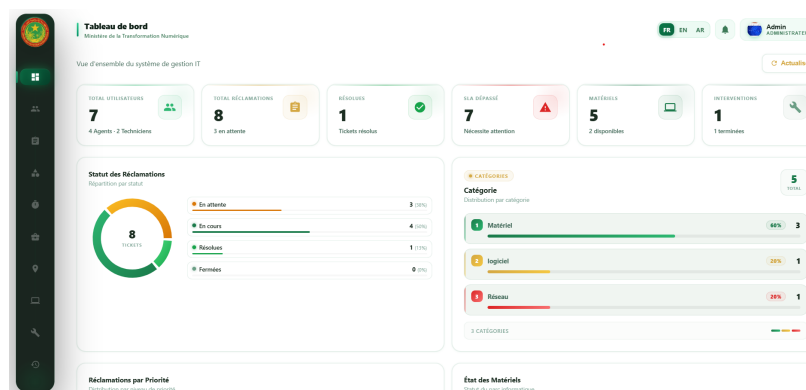


Fig. IV.2 : Page Tableau de bord

Cette interface constitue le point d'entrée principal de l'administrateur après connexion. Elle offre une vue synthétique de l'activité globale du système à travers des indicateurs clés (nombre d'agents, de techniciens, de réclamations par statut) ainsi que des graphiques

de répartition par catégorie. Ce tableau de bord permet à l'administrateur de superviser rapidement l'état général de la plateforme et de prioriser ses actions de gestion.

## Page Réclamations :

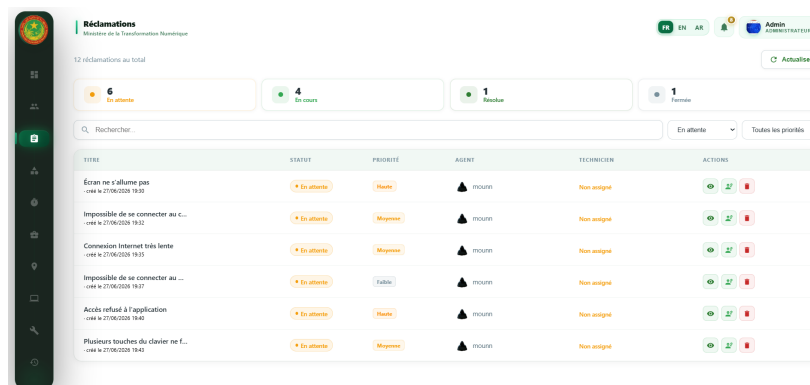


Fig. IV.3 : Page Réclamations :

Cette capture présente la liste complète des réclamations soumises par les agents, avec pour chacune son statut, sa priorité, l'agent déclarant et le technicien assigné. Des filtres de recherche et de tri facilitent le repérage rapide d'une réclamation spécifique. Cette interface constitue l'outil central permettant à l'administrateur d'affecter les incidents aux techniciens et d'en suivre le traitement.

## IV.4 Interface Technicien

### Page Tableau de bord :

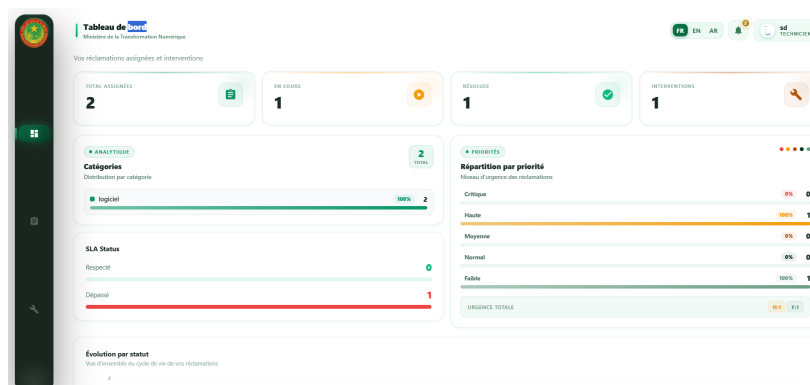


Fig. IV.4 : Page Tableau de bord

Cette interface est dédiée au suivi des interventions assignées à un technicien donné. Elle affiche le nombre de réclamations en attente, en cours et résolues, complété par une

répartition par catégorie et par niveau de priorité (SLA). Ce tableau de bord permet au technicien d'organiser efficacement sa charge de travail quotidienne.

## IV.5 Interfaces Agent

### Page Tableau de bord :

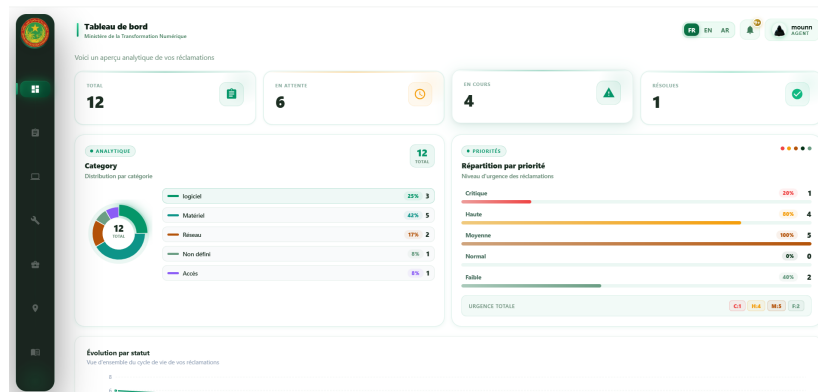


Fig. IV.5 : Page Tableau de bord

Cette capture illustre l'espace de suivi personnel mis à disposition de l'agent. Il y visualise le nombre total de réclamations qu'il a soumises, réparties selon leur statut (en attente, en cours, résolues), ainsi qu'une répartition graphique par catégorie et par priorité. Cette interface offre à l'agent une visibilité claire sur l'avancement de ses demandes.

### Page Mes Réclamations :

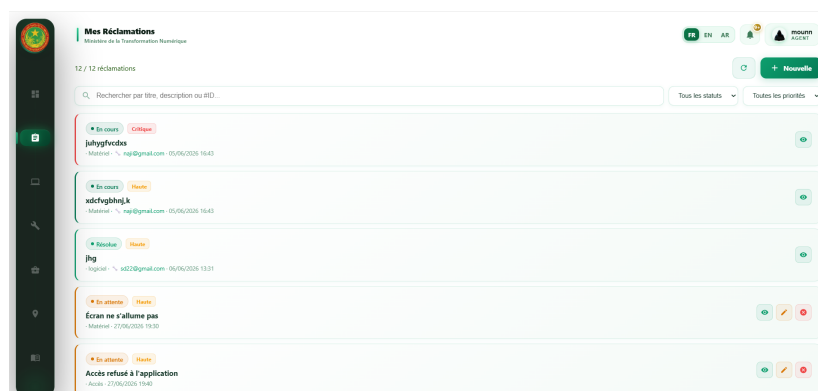


Fig. IV.6 : Page Mes Réclamations

Cette interface liste l'ensemble des réclamations déposées par l'agent connecté, avec pour chacune un indicateur de statut et de priorité ainsi qu'un horodatage. Une barre de recherche et un bouton de création rapide permettent respectivement de retrouver une

*réclamation existante ou d'en signaler une nouvelle. Cette page illustre concrètement le parcours utilisateur de l'agent au sein de la plateforme.*

## **Conclusion**

En conclusion, ce chapitre de réalisation a marqué la phase la plus concrète de notre projet, où les idées issues de l'analyse et de la conception ont été traduites en une application fonctionnelle et opérationnelle. Grâce aux langages et outils choisis, nous avons pu développer les modules essentiels du système, en tenant compte des besoins des différents utilisateurs. Chaque étape de l'implémentation a été une opportunité d'approfondir nos compétences en développement. Nous avons également accordé une attention particulière à la sécurité, à l'ergonomie, ainsi qu'à la clarté du code, afin d'assurer la fiabilité et la maintenabilité du système. Ce travail, réalisé en autonomie, nous a permis de relever plusieurs défis techniques et organisationnels, et de mieux appréhender les contraintes du développement d'un projet réel dans un contexte proche du monde professionnel.

# Conclusion Générale

Ce mémoire s'inscrit dans le cadre du développement d'une **plateforme web de gestion des réclamations IT et du matériel informatique** au sein du Ministère de la Transformation Numérique, de l'Innovation et de la Modernisation de l'Administration (MTNIMA). Face aux insuffisances d'un système de signalement informel et non traçable, l'objectif principal consistait à concevoir et à mettre en œuvre une solution numérique centralisée, fiable et adaptée aux besoins réels de l'administration publique. Au terme de ce projet, les objectifs fixés ont été atteints dans leur ensemble : le système développé centralise la déclaration, l'attribution et le suivi des incidents techniques, en offrant à chaque profil utilisateur (agent, technicien, administrateur) une interface adaptée à ses responsabilités, et intègre notamment un module d'authentification sécurisé par JWT avec gestion des rôles, un système de déclaration et de suivi des réclamations IT, un mécanisme d'attribution des incidents aux techniciens, un tableau de bord statistique pour le pilotage de l'activité, ainsi qu'un historique traçable de toutes les actions effectuées sur chaque réclamation. Tout au long de ce travail, les bonnes pratiques du développement logiciel ont été appliquées, en utilisant **Spring Boot** pour le backend, **React (Vite)** pour le frontend et une base de données **MySQL** avec une authentification sécurisée par **JWT**. Malgré les résultats satisfaisants obtenus, certaines limites ont été identifiées : l'application n'est pas encore déployée en production dans un environnement réseau institutionnel réel, la gestion du matériel informatique reste partielle et pourrait être enrichie, l'absence de notifications en temps réel (WebSocket ou push) limite la réactivité du système, et la couverture des tests automatisés est insuffisante. Parmi les perspectives d'évolution envisagées, on peut citer le développement d'une application mobile Android/iOS permettant de déclarer et de suivre les réclamations en situation de mobilité, l'intégration d'un tableau de bord analytique avancé avec des indicateurs de performance et des rapports exportables automatiquement, la mise en place d'une base de connaissances proposant des suggestions de résolution automatique pour les incidents récurrents, l'ajout de notifications en temps réel par e-mail et push à chaque changement de statut, ainsi que l'intégration au système d'information de l'administration via une connexion aux annuaires LDAP/Active Directory pour une gestion centralisée des comptes. En somme, ce projet constitue une base solide pouvant être facilement enrichie par de nouvelles fonctionnalités au fur et à mesure des besoins réels du terrain. Nous espérons que ce travail contribuera positivement à la modernisation des processus administratifs et qu'il servira de fondement à des évolutions futures plus ambitieuses.

# Références bibliographiques

1. **Pivotal / VMware.** *Spring Boot Reference Documentation*. [En ligne] Disponible sur : <https://docs.spring.io/spring-boot/> (consulté en 2026).
2. **Meta (Facebook).** *React — Documentation officielle*. [En ligne] Disponible sur : <https://react.dev/> (consulté en 2026).
3. **Vite.** *Vite — Next Generation Frontend Tooling*. [En ligne] Disponible sur : <https://vitejs.dev/> (consulté en 2026).
4. **Oracle.** *MySQL 8.0 Reference Manual*. [En ligne] Disponible sur : <https://dev.mysql.com/doc/> (consulté en 2026).
5. **IETF.** *RFC 7519 — JSON Web Token (JWT)*. [En ligne] Disponible sur : <https://datatracker.ietf.org/doc/html/rfc7519> (consulté en 2026).
6. **OMG.** *Unified Modeling Language (UML) Specification*. [En ligne] Disponible sur : <https://www.omg.org/spec/UML/> (consulté en 2026).
7. **Axios.** *Axios — Promise based HTTP client*. [En ligne] Disponible sur : <https://axios-http.com/docs/intro> (consulté en 2026).
8. **React Router.** *React Router — Documentation officielle*. [En ligne] Disponible sur : <https://reactrouter.com/> (consulté en 2026).
9. **Iden, J., & Eikebrokk, T. R.** (2013). *Implementing IT Service Management : A systematic literature review*. International Journal of Information Management, 33(3), 512–523.

# Annexes

## Annexe A — Diagramme entité-association (MCD)

Le diagramme entité-association (MCD) ci-dessous représente la structure des données de la plateforme **GesRecIT**. Il modélise les entités principales du système ainsi que les associations qui les relient.

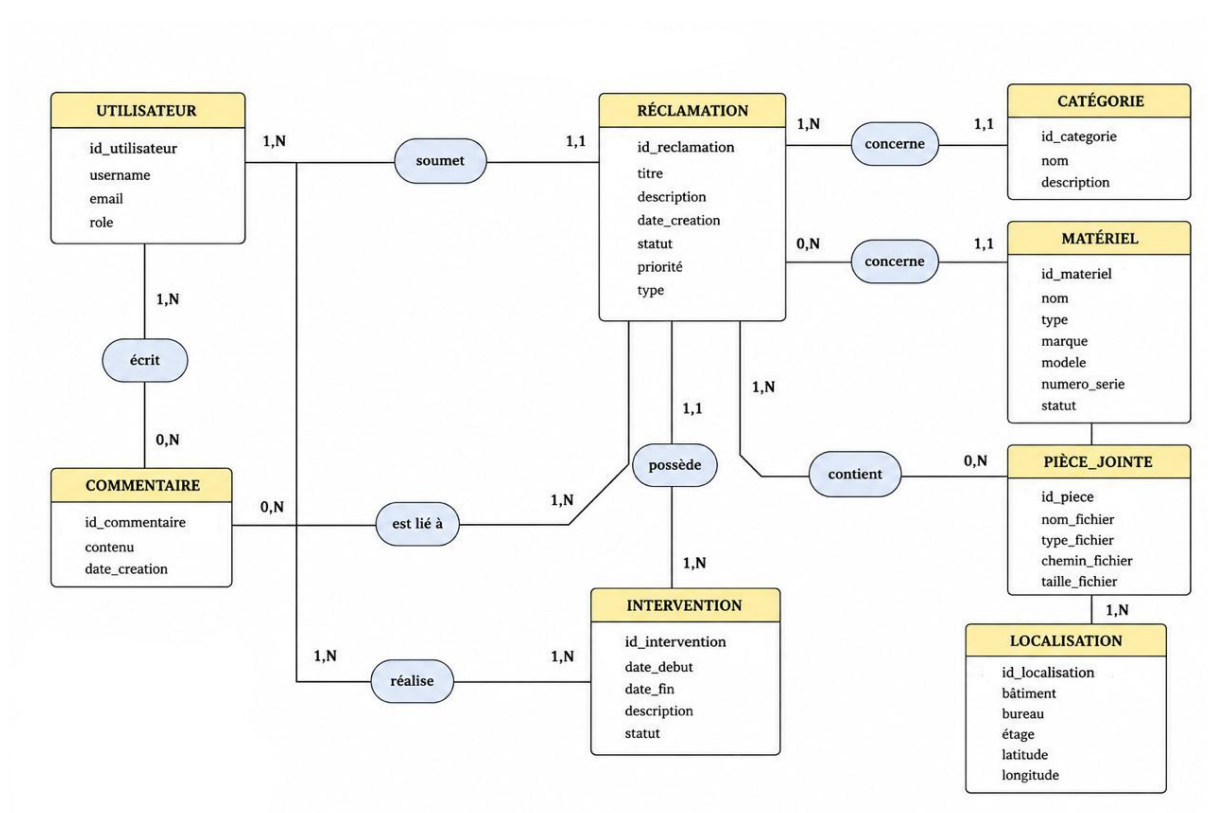


Fig. IV.7 : Modèle Conceptuel des Données (MCD) — GesRecIT

## Annexe B — Modèle logique des données (MLD)

Le modèle logique des données décrit la structure relationnelle de la base MySQL. Les tables principales sont présentées ci-après :

| Table                           | Description                                                           |
|---------------------------------|-----------------------------------------------------------------------|
| <code>utilisateurs</code>       | Stocke les informations des agents, techniciens et administrateurs    |
| <code>reclamations</code>       | Enregistre chaque réclamation IT avec son statut et sa catégorie      |
| <code>categories</code>         | Définit les types de réclamations (Matériel, Logiciel, Réseau, Accès) |
| <code>affectations</code>       | Trace l'attribution d'une réclamation à un technicien                 |
| <code>interventions</code>      | Décrit les actions techniques réalisées sur une réclamation           |
| <code>materiels</code>          | Inventaire du parc informatique géré par l'administration             |
| <code>historique_statuts</code> | Conserve l'historique complet des changements de statut               |
| <code>commentaires</code>       | Échanges textuels entre acteurs autour d'une réclamation              |
| <code>pieces_jointes</code>     | Fichiers joints à une réclamation (images, documents)                 |

TAB. IV.1 Tables principales de la base de données GesRecIT

## Annexe C — Guide d'installation et de déploiement

### Prérequis

- Java 17 ou supérieur (JDK)
- Node.js 18 ou supérieur (avec npm)
- MySQL 8.0 ou supérieur
- Maven 3.8+

### Étapes d'installation

1. **Cloner le dépôt** : récupérer le code source depuis le dépôt Git du projet.
2. **Configurer la base de données** : créer une base MySQL `gesrecit_db` et exécuter le script SQL fourni (`init.sql`).
3. **Configurer le backend** : modifier `application.properties` avec les identifiants de la base de données et la clé secrète JWT.
4. **Lancer le backend** : exécuter `mvn spring-boot:run` depuis le répertoire `backend/`.
5. **Lancer le frontend** : exécuter `npm install` puis `npm run dev` depuis le répertoire `frontend/`.
6. **Accéder à l'application** : ouvrir `http://localhost:5173` dans un navigateur web.

## Annexe D — Extrait du code source

### Authentication JWT — Backend (Spring Boot)

Listing IV.1 Filtre d'authentification JWT

```
1 @Component
2 public class JwtAuthFilter extends OncePerRequestFilter {
3
4     @Autowired
5     private JwtService jwtService;
6
7     @Override
8     protected void doFilterInternal(HttpServletRequest request,
9         HttpServletResponse response, FilterChain filterChain)
10        throws ServletException, IOException {
11
12        String authHeader = request.getHeader("Authorization");
13        if (authHeader == null || !authHeader.startsWith("Bearer ")) {
14            filterChain.doFilter(request, response);
15            return;
16        }
17        String token = authHeader.substring(7);
18        String username = jwtService.extractUsername(token);
19        if (username != null && SecurityContextHolder
20            .getContext().getAuthentication() == null) {
21            // Valider et injecter l'authentification dans le contexte
22            UserDetails userDetails =
23                userDetailsService.loadUserByUsername(username);
24            if (jwtService.isTokenValid(token, userDetails)) {
25                UsernamePasswordAuthenticationToken authToken =
26                    new UsernamePasswordAuthenticationToken(
27                        userDetails, null, userDetails.getAuthorities());
28                SecurityContextHolder.getContext()
29                    .setAuthentication(authToken);
30            }
31        }
32        filterChain.doFilter(request, response);
33    }
34 }
```

## Annexe E — Attestation de stage

الجمهورية الإسلامية الموريتانية  
شرف - إخاء - عدل

RÉPUBLIQUE ISLAMIQUE DE MAURITANIE  
Honneur - Fraternité - Justice

وزارة التحول الرقمي وعصرنة الإدارة  
Ministère de la Transformation Numérique et de la  
Modernisation de l'Administration  
مديرية التطوير والتشغيل البيئي  
Direction du Développement et de l'interopérabilité  
المدير  
Le Directeur

نواكشوط، في 10 3 2026  
Nouakchott, le 10 3 2026  
الرقم 1000255  
Numéro 1000255

ATTESTATION DE STAGE

Je, soussigné Monsieur **Mohamed Ould Boba**, Directeur du Développement et de l'Interopérabilité au Ministère de la Transformation Numérique et de la Modernisation de l'Administration.

Atteste que :

L'étudiante **Mouna Naji**, inscrite en 3<sup>e</sup> année de Licence en Méthodes Informatiques Appliquées à la Gestion des Entreprises (MIAGE) à la Faculté des Sciences et Techniques (FST) a effectué un stage au sein de notre direction, sous le thème : Conception et développement d'une plateforme web de gestion des réclamations IT et du matériel informatique, durant la période **du 09 Février au 30 Juin 2026**.

En foi de quoi, la présente attestation lui est délivrée pour servir et valoir ce que de droit.

Mohamed Ould Boba

Le Directeur

نواكشوط - موريتانيا  
Nouakchott - Mauritanie • mt.nima.gov.mr

## Annexe F — Comptes personnels

Les comptes professionnels et personnels suivants permettent de suivre l'actualité et les travaux de l'auteure de ce mémoire :

- **LinkedIn** : Mouna Naji
- **Facebook** : Mouna Naji
- **GitHub** : mounanaji810-star